

When Deep Q-Network Scheduling Becomes a Dispatching Rule: A Reproducible Diagnostic Study of Dynamic Job Shops

Author Name

Department or School

Institution Name

City, Country

author.email@example.com

Abstract—Deep reinforcement learning is increasingly used to select dispatching rules in dynamic job shop scheduling, yet aggregate scheduling scores can conceal what a learned policy actually does. This paper presents a reproducible restoration and diagnostic evaluation of a dueling Double Deep Q-Network (DQN) scheduler whose actions are nine classical dispatching rules. The implementation is modularized, tested, and evaluated with validation-selected checkpoints on two complementary test beds: 81 held-out generated dynamic instances spanning three job counts, due-date tightness levels, and arrival settings, and 60 dynamic variants derived from 15 named OR-Library job shop instances. Three independent DQN training seeds are compared transparently with all nine rules. On generated instances, DQN ranks second with mean terminal tardiness rate 0.3402, behind shortest processing time (SPT) at 0.3239. On benchmark-derived instances, DQN again ranks second at 0.5247, behind SPT at 0.5148 and slightly ahead of apparent tardiness cost (ATC) at 0.5288. Policy traces explain these results: two checkpoints select SPT for every decision, while the third selects most remaining processing time (MRT) for every decision, across both test beds. Thus, the selected agents do not exhibit adaptive rule mixing. The study shows that policy-level diagnostics are necessary before attributing heuristic performance to learned scheduling intelligence and establishes a reproducible baseline for future regime-aware policies.

Index Terms—dynamic job shop scheduling, deep reinforcement learning, DQN, dispatching rules, policy diagnostics, reproducibility

I. Introduction

Dynamic job shop scheduling (DJSS) requires decisions while jobs arrive and the shop state changes. Priority dispatching rules remain attractive because they are fast, interpretable, and reactive, but their relative performance depends on congestion, routing, due-date allowance, and the optimization criterion [1], [2]. Deep reinforcement learning (DRL) offers an appealing alternative: a policy can observe the current shop and select a scheduling action without solving a new mathematical program at each event [3], [4].

There is, however, a distinction between using a neural network and learning an adaptive scheduler. In rule-selection formulations, a network chooses from a fixed menu of dispatching rules. A strong aggregate result may

therefore arise because the network has discovered that one rule is globally reliable, rather than because it switches rules in response to the state. This distinction matters scientifically and operationally. A static rule has lower implementation cost, is easier to audit, and should not be replaced by an opaque network unless the network provides a measurable adaptive benefit.

DRL results are also sensitive to random seeds, model selection, and experimental protocol [5]. A legacy notebook that executes once on a single instance cannot support claims about generalization. The work reported here began as a reproducibility restoration of such a notebook and was extended into a held-out, multi-seed diagnostic study.

The contributions are:

- a reproducible DJSS package that separates simulation, data generation, training, evaluation, benchmark conversion, statistical summaries, and policy tracing;
- a validation-selected, three-seed evaluation on 81 held-out generated instances and 60 OR-Library-derived dynamic instances, with all nine dispatching rules reported;
- action-level evidence that all three selected DQN checkpoints collapse to deterministic single-rule policies, explaining both their competitiveness and their failure to outperform SPT; and
- a reviewer-safe interpretation of DQN rule selection as a reproducible baseline and diagnostic result, rather than evidence of broad superiority over classical rules.

II. Related Work

A. Dispatching Rules for Dynamic Scheduling

Dispatching rules map local shop information to a priority ordering. SPT favors the smallest immediate processing time; earliest due date prioritizes the nearest due date; slack- and critical-ratio rules combine time, due dates, and remaining work. ATC adds look-ahead structure for tardiness-oriented scheduling [2]. Comparative simulation has long shown that no simple rule dominates every dynamic shop configuration or objective [1]. More recent

work uses empirical learning to design interpretable rules that generalize across dynamic regimes [6].

B. Deep Reinforcement Learning for Job Shops

DQN combines value-function approximation with replay and a target network [7]. Double Q-learning reduces maximization bias [8], a dueling network separates state value from action advantage [9], and prioritized replay samples transitions according to temporal-difference error [10]. These components appear in many scheduling implementations.

Two broad DRL scheduling formulations are common. Operation-level policies construct a schedule directly, often using graph representations [4]. Rule-selection policies instead choose among handcrafted dispatching rules. Luo [3] used DQN to select composite rules under new job insertions, while PPO-based work has considered multi-agent manufacturing and uncertain processing times [11], [12]. Recent comparative evidence indicates that carefully chosen state and action sets can make dynamic rule selection effective, but also that periodic selection may be more robust than real-time reinforcement learning [13]. PPO [14] is a natural future candidate when a stochastic policy and explicit entropy control are desired.

The present paper asks a narrower question than these algorithmic studies: when a restored DQN scheduler appears competitive, is it actually adapting its rule choice? This question is answered by tracing every greedy action selected by every retained checkpoint.

III. Problem and MDP Formulation

A. Dynamic Job Shop

Let $\mathcal{J}$ be a set of jobs. Job j arrives at release time a_j and contains an ordered sequence of non-preemptive operations $\mathcal{O}_j$. Each operation has one or more compatible machines and a machine-dependent processing time. A machine processes at most one operation at a time, an operation is scheduled only after its predecessor has completed, and scheduling decisions occur when an idle machine has legal candidate operations.

The due date is generated as

$$d_j = a_j + \delta P_j, \quad (1)$$

where P_j is the sum of average operation processing times and δ is the due-date tightness (DDT) parameter. Smaller δ produces tighter due dates.

The implementation reports a terminal tardiness rate

$$\tau = \frac{\sum_{j \in \mathcal{J}} n_j^{\text{tardy}}}{\sum_{j \in \mathcal{J}} |\mathcal{O}_j|}, \quad (2)$$

where n_j^{tardy} is the number of operations marked tardy for job j after its due date is crossed. Lower τ is better. This operation-based rate is the primary outcome throughout the paper; it should not be confused with total tardiness in time units.

TABLE I
Dispatching-Rule Action Space

Action	Priority	Action	Priority
MRT	most remaining work	MCR	minimum job CR
LRT	least remaining work	CR	minimum operation CR
SPT	shortest processing time	ATC	apparent tardiness cost
EDD	earliest due date	LSPO	least slack per operation
SLK	least job slack		

B. State, Actions, and Reward

The non-maintenance observation contains 14 continuous features. They include the inverse arrival parameter, DDT, job count, average current machine completion time, buffer completion rate, job- and operation-level actual and expected tardiness rates, mean and standard deviation of completion by operation count and processing time, and the minimum critical ratio among candidate jobs.

The action $a_t \in \{1, \dots, 9\}$ selects one dispatching rule for the current ready machine. Table I lists the menu. The selected rule then chooses the operation from that machine's legal candidates.

At each decision, the environment estimates actual and expected local tardiness, denoted $\hat{\tau}_t$. The dense reward is

$$r_t = \hat{\tau}_{t-1} - \hat{\tau}_t. \quad (3)$$

The reward is positive when the estimated tardiness rate decreases and negative when it increases.

IV. DQN and Reproducibility Protocol

A. Agent

The agent uses a dueling Double DQN. A seven-layer multilayer perceptron has widths [207, 145, 78, 79, 205, 105, 217]. Each hidden layer is followed by layer normalization, Leaky ReLU, and dropout with probability 0.2. The final representation feeds a scalar value stream $V(s)$ and a nine-dimensional advantage stream $A(s, a)$:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'). \quad (4)$$

For transition (s_t, a_t, r_t, s_{t+1}) , the Double DQN target is

$$y_t = r_t + \gamma Q^-(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a')), \quad (5)$$

with $\gamma = 0.99$. The replay buffer holds 100,000 transitions. Prioritized replay uses $\alpha = 0.6$, importance-sampling exponent $\beta = 0.4$, and a batch size of 32. Learning starts after 500 transitions. Adam uses learning rate 10^{-3} and weight decay 10^{-5} ; gradients are clipped to unit norm. The target network receives a soft update with $\tau = 0.01$ every 1,000 learning steps.

Exploration begins at $\epsilon = 1.0$, decays by 0.995 after each gradient update, and is bounded below by 0.01. Evaluation is greedy with $\epsilon = 0$. Three independent training seeds,

11, 22, and 33, are run for 500 episodes each. Every episode samples one of the generated training instances. Every 50 episodes, the agent is evaluated on the complete validation split, and the checkpoint with minimum mean validation tardiness is retained. The test split is not used for checkpoint selection.

B. Generated Instance Matrix

The generated matrix varies jobs in $\{20, 50, 100\}$, DDT in $\{0.5, 1.0, 1.5\}$, and mean interarrival parameter in $\{50, 100, 200\}$. Every shop has 15 machines arranged in five work centers. Jobs contain 6–10 operations, processing times are sampled from 60–120 time units, and each operation is compatible with one or more machines. Half of the jobs are initially available; remaining arrivals follow exponential interarrival times.

Independent instance seeds create 54 training instances (seeds 101 and 202), 54 validation instances (505 and 606), and 81 held-out test instances (707, 808, and 909). Thus, both random instance realization and DQN initialization are separated across the experimental stages.

C. OR-Library-Derived Matrix

OR-Library supplies established job shop routing and processing-time instances [15]. Fifteen named instances are used: ft06, ft10, ft20, la01, la06, la21, la31, orb01, orb02, abz5, abz7, swv01, swv06, yn1, and yn2. Each is converted into four dynamic variants using DDT $\{0.5, 1.0\}$ and interarrival parameters $\{50, 100\}$, yielding 60 datasets. Half of the jobs are released initially.

These are deliberately described as benchmark-derived, not as standard OR-Library benchmark results. The conversion preserves named routing and processing times but adds stochastic release times and due dates, changing both the problem and objective from the original static makespan setting.

D. Evaluation and Statistics

All nine fixed rules and all three greedy DQN checkpoints run in the same event simulator. For each method, the report includes mean, median, interquartile range, sample standard deviation, and a normal-approximation 95% confidence interval. DQN-minus-baseline differences are paired by checkpoint and instance. Zero differences are omitted from a two-sided Wilcoxon signed-rank test, and paired rank-biserial correlation is reported as an effect size. Negative differences favor DQN.

Because each fixed-rule result is reused for three DQN checkpoints on the same instance, the aggregate Wilcoxon rows are not a hierarchical analysis of independent training runs. Their p -values are treated as descriptive evidence. The per-seed results and action traces are the primary evidence about training variability.

TABLE II
Generated Held-Out Test Results

Method	n	Mean τ	Median	95% CI
SPT	81	0.3239	0.3337	0.0527
DQN	243	0.3402	0.3582	0.0304
MRT	81	0.3727	0.4086	0.0524
ATC	81	0.3786	0.4196	0.0613

TABLE III
OR-Library-Derived Test Results

Method	n	Mean τ	Median	95% CI
SPT	60	0.5148	0.5767	0.0459
DQN	180	0.5247	0.5833	0.0260
ATC	60	0.5288	0.6000	0.0472
MRT	60	0.5445	0.5967	0.0438

V. Results

A. Generated Held-Out Instances

Table II reports the four strongest methods on the generated test matrix. SPT is first with mean tardiness 0.3239. Aggregate DQN is second at 0.3402, followed by MRT and ATC. DQN is therefore competitive but does not improve on the strongest fixed rule.

The aggregate DQN-minus-SPT difference is $+0.0163$ over 243 checkpoint-instance pairs (two-sided Wilcoxon $p = 1.23 \times 10^{-11}$; rank-biserial $r = 0.932$). DQN records 9 wins, 61 losses, and 173 ties against SPT. Against ATC, DQN's mean difference is -0.0384 ($p = 6.90 \times 10^{-12}$; $r = -0.542$), with 141 wins, 72 losses, and 30 ties.

The large number of ties against SPT already suggests that more than one checkpoint may reproduce it exactly. Regime-level differences in Fig. 1 show that aggregate DQN remains slightly worse than SPT for every tested job count, DDT level, and arrival setting. The largest gap occurs for 50-job instances and for loose due dates ($\delta = 1.5$).

B. Benchmark-Derived Generalization

Table III shows the benchmark-derived results. The ordering is consistent with the generated matrix: SPT ranks first at 0.5148, DQN second at 0.5247, ATC third at 0.5288, and MRT fourth at 0.5445. Figure 2 summarizes the agreement across the two test beds.

Against SPT, DQN has a mean difference of $+0.0099$ over 180 pairs ($p = 5.12 \times 10^{-6}$; $r = 0.720$), with 13 wins, 40 losses, and 127 ties. Against ATC, the difference is -0.0041 ($p = 0.00169$; $r = -0.302$), with 98 wins, 46 losses, and 36 ties. DQN is slightly better than ATC on average, but the effect is small relative to the gap over weaker baselines.

The study records 718 successful rows from 720 scheduled method-instance evaluations. Two LRT rows on arrival-50 variants of abz7 reach a no-event deadlock. The simulator reports these failures explicitly rather than hanging or silently discarding them.

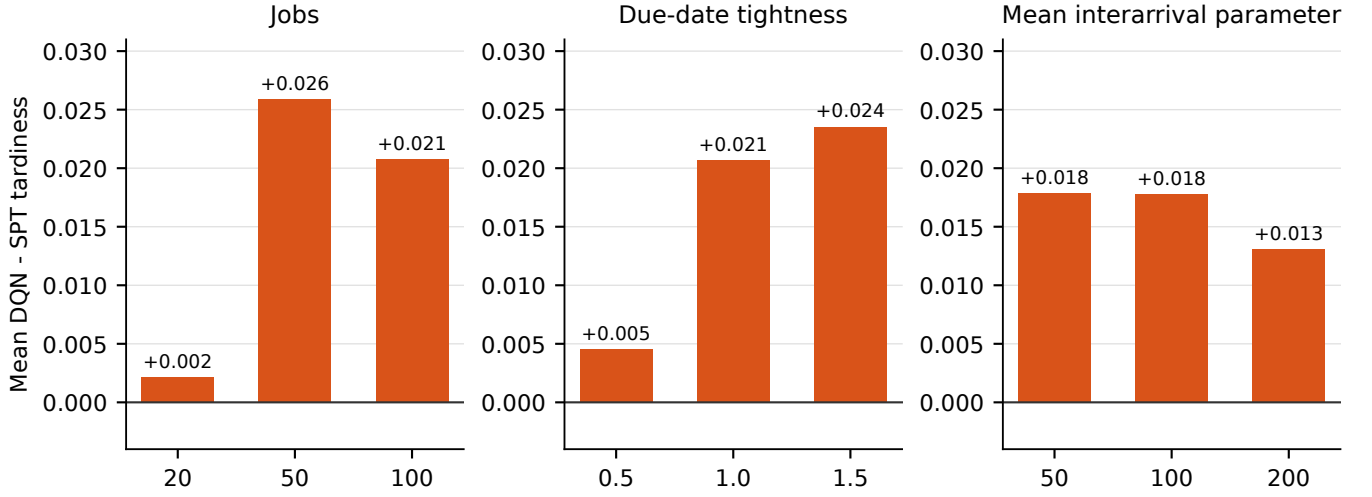


Fig. 1. Mean DQN-minus-SPT tardiness on generated held-out instances. Positive values favor SPT.

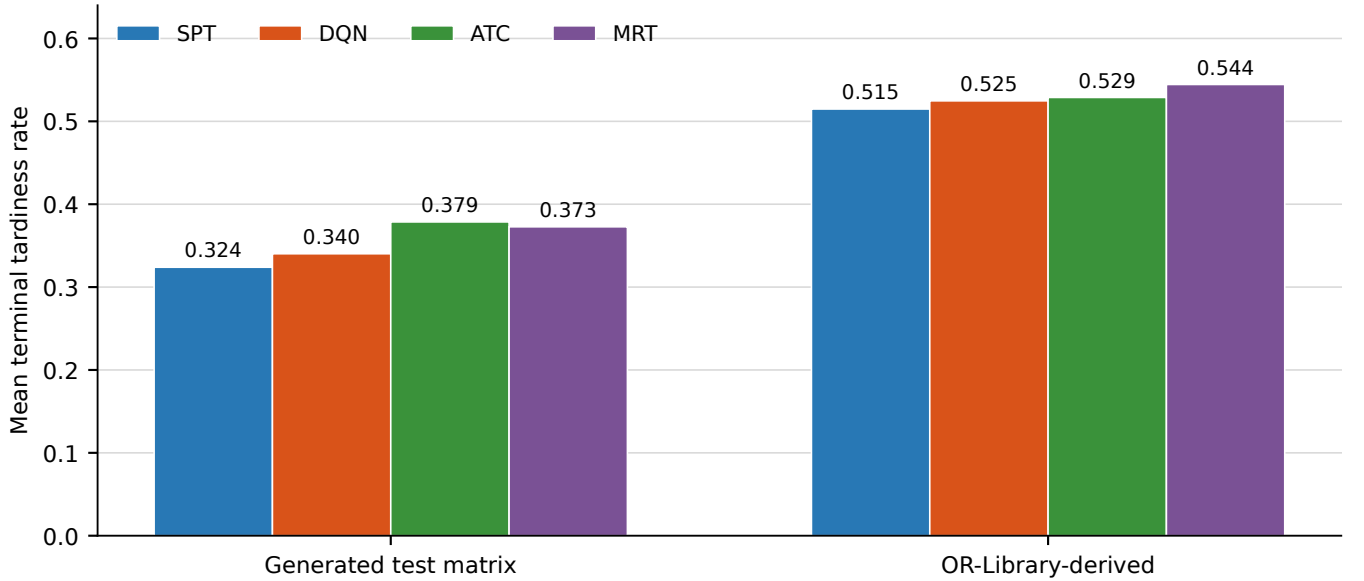


Fig. 2. Mean terminal tardiness across the two held-out studies. Lower is better.

C. Policy-Trace Diagnosis

The aggregate means conceal a simple policy structure. As Table IV and Fig. 3 show, seeds 11 and 22 select SPT for 100% of decisions, while seed 33 selects MRT for 100%. This behavior is identical on generated and benchmark-derived datasets. The seed-level terminal tardiness means consequently equal those of the copied rules: 0.3239 for seeds 11 and 22 and 0.3727 for seed 33 on generated data; 0.5148, 0.5148, and 0.5445 on benchmark-derived data.

These traces establish that the DQN does not learn regime-aware rule mixing under the tested protocol. Its second-place aggregate rank is the average of two SPT policies and one MRT policy. Describing this behavior only

TABLE IV
Greedy Checkpoint Policy Traces

Test bed	Seed	Decisions	Dominant action	Fraction
Generated	11	37,227	SPT	1.000
Generated	22	37,227	SPT	1.000
Generated	33	37,374	MRT	1.000
Derived	11	10,844	SPT	1.000
Derived	22	10,844	SPT	1.000
Derived	33	10,844	MRT	1.000

as “DQN performance” would obscure the mechanism.

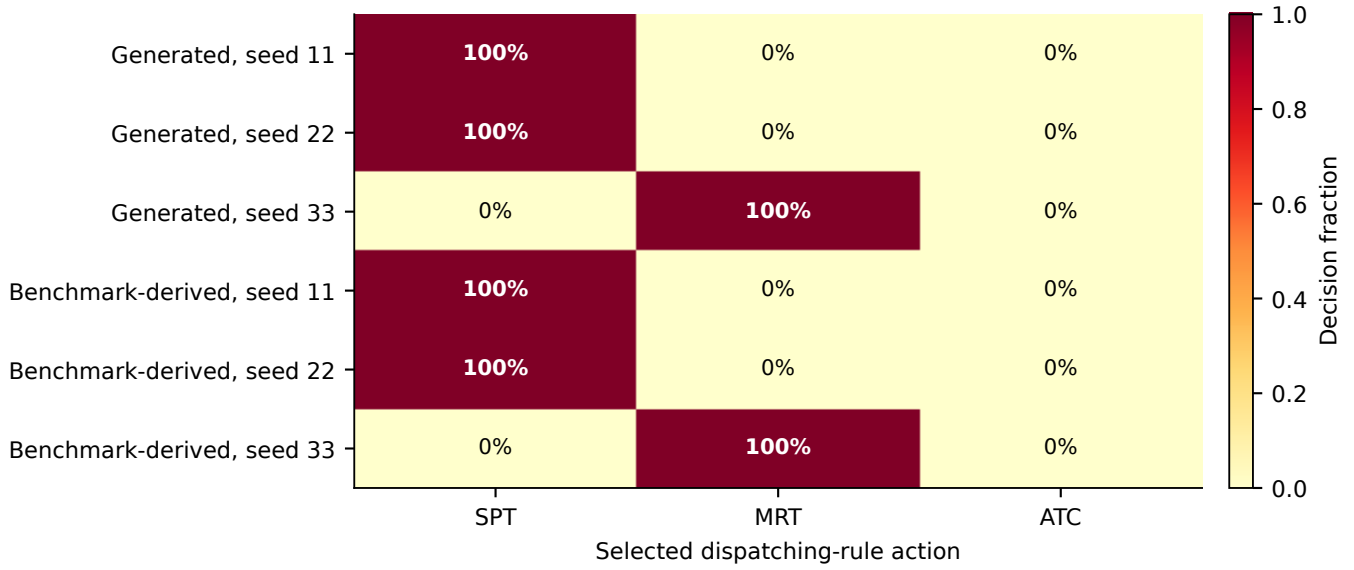


Fig. 3. Action fractions for each greedy checkpoint. Every checkpoint collapses to one classical dispatching rule.

VI. Discussion

A. What the Agent Learned

The policy collapse has two interpretations. Positively, DQN identifies two strong rules without being explicitly told which action is best, and two training seeds recover the best broad baseline. This demonstrates that the end-to-end training and validation pipeline is functional. Negatively, the neural policy adds no adaptive value at deployment: the retained checkpoints can be replaced by SPT or MRT with identical decisions and greater transparency.

The collapse is plausible given the formulation. Each action delegates the scheduling decision to a complete handcrafted heuristic. A value learner can obtain reliable returns by assigning one action a globally dominant Q-value. The exploration schedule reaches its minimum rapidly because decay occurs after gradient updates, and deterministic validation selection rewards the most consistently strong action. Dense reward improves credit assignment but does not itself encourage policy diversity or state-dependent switching.

B. Implications for Evaluation

Three reporting practices follow from this result. First, all strong fixed rules, including SPT and ATC, must remain in the main comparison. Removing an inconvenient baseline would make the learned method appear stronger without changing its behavior. Second, checkpoint selection must use a validation split rather than the final test matrix. Third, a rule-selection paper should report action distributions by seed and regime. Aggregate reward curves and final tardiness scores are insufficient when a learned policy can exactly copy one baseline.

This framing is useful even though it is a negative algorithmic result. Reproducible evidence that a complex learner reduces to a simple rule prevents false attribution, identifies where model capacity is not being used, and supplies a strong baseline for subsequent methods.

C. Path Toward Adaptive Policies

A stronger scheduler should demonstrate state-dependent action changes that improve over the best single rule on independent test regimes. One candidate is a regime-aware PPO design (RA-PPO) with a stochastic actor, entropy regularization, and explicit state features for congestion and due-date pressure. PPO’s clipped objective supports repeated minibatch updates while limiting policy movement [14]. The key test is not whether RA-PPO uses more than one action during exploration, but whether its greedy or low-temperature deployment policy retains meaningful action diversity and yields a statistically and practically significant gain over SPT.

Recommended ablations include state-feature groups, dense versus sparse reward, entropy coefficient, checkpoint criterion, training budget, and the removal of each action rule. Evaluation should use the same generated and benchmark-derived matrices reported here so that improvements cannot be attributed to an easier test set.

VII. Limitations

The study has six main limitations. First, only three DQN training seeds and 500 episodes per seed were used; this is adequate to expose collapse but not to characterize the full tail of training variability. Second, the expanded study evaluates one dense-reward configuration rather than a complete hyperparameter and reward ablation. Third, the OR-Library instances are dynamically

transformed; the results are not comparable to published static makespan values. Fourth, the aggregate paired tests reuse each baseline outcome across three checkpoints and therefore do not replace a hierarchical seed-by-instance analysis. Fifth, the action space is restricted to nine existing rules, so the agent cannot construct a novel operation-level priority function. Finally, there is no real-factory, manufacturing execution system, or digital-twin validation.

The two explicit LRT deadlocks also show that simulator robustness is part of the experimental result. Future work should diagnose and eliminate these no-event states before expanding the benchmark family.

VIII. Conclusion

This paper restored a legacy DQN DJSS implementation into a tested, reproducible experiment package and evaluated it on independent generated and OR-Library-derived dynamic instances. DQN was competitive and ranked second in both studies, but it did not outperform SPT. More importantly, action traces showed that every retained checkpoint selected a single classical rule for every decision: two copied SPT and one copied MRT. The central conclusion is therefore diagnostic rather than promotional. In dispatching-rule selection, policy tracing is necessary to distinguish adaptive scheduling from neural imitation of a fixed heuristic. The released protocol provides a transparent baseline for future policies that claim regime-aware rule mixing and stronger generalization.

Artifact Availability

The source code, configurations, converted benchmark data, checkpoints, raw CSV results, tests, and policy traces are available at <https://github.com/PinkieChi/djss-rl-cleanup>. The manuscript figures are generated directly from the committed CSV files.

References

[1] C. Rajendran and O. Holthaus, “A comparative study of dispatching rules in dynamic flowshops and jobshops,” *Eur. J. Oper. Res.*, vol. 116, no. 1, pp. 156–170, 1999.

[2] A. P. J. Vepsalainen and T. E. Morton, “Priority rules for job shops with weighted tardiness costs,” *Manage. Sci.*, vol. 33, no. 8, pp. 1035–1047, 1987.

[3] S. Luo, “Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning,” *Appl. Soft Comput.*, vol. 91, p. 106208, 2020.

[4] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, “Learning to dispatch for job shop scheduling via deep reinforcement learning,” in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1621–1632. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/11958dfee29b6709f48a9ba0387a2431-Abstract.html>

[5] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3207–3214.

[6] C. Ferreira, G. Figueira, and P. Amorim, “Effective and interpretable dispatching rules for dynamic job shops via guided empirical learning,” *Omega*, vol. 111, p. 102643, 2022.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[8] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.

[9] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling network architectures for deep reinforcement learning,” in *Proc. 33rd Int. Conf. Mach. Learn.*, ser. *Proc. Mach. Learn. Res.*, vol. 48, 2016, pp. 1995–2003. [Online]. Available: <https://proceedings.mlr.press/v48/wangf16.html>

[10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Proc. 4th Int. Conf. Learn. Represent.*, 2016. [Online]. Available: <https://arxiv.org/abs/1511.05952>

[11] Y. Zhang, H. Zhu, D. Tang, T. Zhou, and Y. Gui, “Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems,” *Robot. Comput.-Integr. Manuf.*, vol. 78, p. 102412, 2022.

[12] X. Wu, X. Yan, D. Guan, and M. Wei, “A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time,” *Eng. Appl. Artif. Intell.*, vol. 131, p. 107790, 2024.

[13] N. Marques, G. Figueira, and L. Guimarães, “Dynamic dispatching rule selection for the job shop scheduling problem,” *Comput. Ind. Eng.*, vol. 210, p. 111471, 2025.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

[15] J. E. Beasley, “OR-library: Distributing test problems by electronic mail,” *J. Oper. Res. Soc.*, vol. 41, no. 11, pp. 1069–1072, 1990.